Virtual Environment

pip

- Download and install packages from PyPI (python Package Index).
- Automatically installs the dependencies required by a package.

```
pip install package_name                    # install a package
pip uninstall package_name                  # uninstall a package
pip install --upgrade nom_du_package        # upgrade a package
pip install --user nom_du_package           # Install a package for the current user
                                                (without admin rights).
pip list                                    # list installed package
pip list –outdated                          # list outdated package
pip check                                   #  Check for dependency conflicts
between                                                 installed packages.
pip freeze > requirements.txt                       # Export the dependancy.
pip install –r requirements.txt             # install from a file
```

pip

By default :

- **On Linux/macOS:** /usr/local/lib/pythonX.Y/site-packages/
- **On Windows:** C:\PythonX.Y\Lib\site-packages\

To avoid!

- Version conflicts between projects (e.g., one project requires Python 3.8, another requires Python 3.10).
- Dependency management (e.g., two projects require different versions of numpy).

Virtual Environments

- It is an isolated folder that contains:
  - A specific Python installation.
  - Libraries (packages) installed independently of the system or other projects.

- Isolate each project's dependencies in a dedicated environment.
- Avoid conflicts and ensure reproducibility.

Venv

- venv is integrated into Python.
- Lightweight and easy to use.

Command (Linux/MacOS)	Commande (Windows)	
python3 -m venv my_env	python -m venv my_env	Create an environnement.
source my_env/bin/activate	my_env\Scripts\activate	Activate the environment.
deactivate	deactivate	Deactivate the environment.

- Does not manage Python versions or non-Python dependencies.

Conda

- Environment and package management tool, mainly used in data science.
- Manages Python and non-Python packages (e.g., gcc, ffmpeg).

```
conda create --name my_env python=3.9    # Create an environment with Python 3.9.
conda activate my_env                    # Activate the environment.
conda deactivate                         # Deactivate environment.
conda install numpy                      # Install a package.
conda env export > environment.yml       # Export the environment.
conda env create -f environment.yml      # Create an environment from a file.
```

- You can also use pip commands with conda

Anaconda vs Miniconda

Anaconda

Complete distribution (+1,500 packages).

Less flexible and heavier.

Miniconda

Lightweight version (only conda and its dependencies).

Lighter, more flexible, and modular.

Venv vs conda

venv

Pure Python projects.

Lightweight environments.

Miniconda

Projects requiring non-Python dependencies.

Advanced management of Python versions and packages.

Additional Tools

- pipenv: Combines pip and venv with a Pipfile to manage dependencies.

- poetry: Modern tool for managing dependencies and virtual environments.

pipenv

- Tool that combines virtual environment and dependency management into a single workflow.
- Distinguishes between development dependencies (de-packages) and production dependencies.

- Easy to use, ideal for pure Python projects.
- Less suitable for non-Python dependencies.
- Can be slow on very large projects.

pipenv

File	Purpose	Managed By	Format
`Pipfile`	Declares top-level project dependencies and constraints	Developers	TOML
`Pipfile.lock`	Records exact versions and hashes of all resolved dependencies	Pipenv automat-ically	JSON

```
pipenv install                          # Install a package and update the Pipfile.
pipenv install --dev                    # Install a development package.
pipenv shell                            # Activate the virtual environment.
pipenv lock                             # generate or update the Pipfile.lock.
pipenv install -r requirements.txt      # Converts a requirements.txt file into a Pipfile.
```

poetry

- Uses a standardised pyroproject.toml file to define dependencies
- Integrated for package publishing
- Less suitable for non-Python dependencies.

poetry add my_package	Add a package to the dependencies.
poetry install	Install all dependencies.
poetry shell	Activates the virtual environment.
poetry update	Update the dependencies.
poetry build	Create a package for publication.

Good practice

- One virtual environment per project (venv, conda, pipenv, poetry)
- No global package installation (Avoid pip install --user or conda install without an environment activated)
- Version dependency files (requirements.txt, Pipfile, environment.yml, pyproject.toml/poetry.lock)
- Document the Python version (.python-version or README)
- Full reproducibility with Docker (Environment + system)

Venv exercice

- Create a folder for a new project.
- In this folder, create a virtual environment with venv.
- Activate the environment.
- Install different packages.
- Check for conflicts with pip check.
- Export dependencies to a requirements.txt file.
- Deactivate the environment.
- Reinstall this environment with the requirements.txt file.

Conda exercices

- Install miniconda
- Create a conda environment with Python 3.9.
- Install different packages.
- Export the environment to environment.yml.
- Delete the environment and recreate it from the environment.yml file.